

MAE014 - Tóp. Ciência Dados B [Turmas MAP e EMA]

Preenchimento de Região em Matriz de Caracteres

Nome completo	DRE
Emanuel Cardoso	122033151
Alanna Figueiredo Simões	120053919
Felipe Rodrigues Schoeffer	123311188
Artur de Melo Brito	117224234

Data: 11/09/2026

Code Review — Preenchimento de Região

Escopo

Compara os dois rascunhos brutos gerados pela IA (reproduzidos em POST_MORTEM.md, seção 3) com a implementação final entregue (preenchimento_pilha.py e preenchimento_recurso.py).

Ponto	Risco na versão ingênua	Correção
Estrutura de posições	Duas Pilha paralelas (linhas/colunas)	Uma Pilha de inteiros codificados (linha*n_colunas+coluna)
Capacidade da pilha	Estimada por excesso (× 4), sem garantia	Capacidade exata n_linhas * n_colunas
Momento da validação	Ao desempilhar (permite reempilhar lixo)	Ao descobrir o vizinho, antes de empilhar
Estrutura da matriz	Lista de strings (imutável)	Lista de listas de caracteres (mutável)
Recurso	Sem proteção — usa o limite padrão do Python	Executada em thread dedicada, com limite e pilha nativa ajustados
Reprodução passo a passo	Acoplada a cada algoritmo	Módulo único (reprodutor.py / interface_grafica.py), independente do algoritmo

Ineficiências e problemas identificados

1. Duas pilhas paralelas em vez de uma

A versão ingênua usa pilha_linhas e pilha_colunas sincronizadas manualmente. Além de duplicar o número de chamadas de empilhar/desempilhar, qualquer divergência entre as duas (ex.: uma exceção no meio de um for) deixaria as pilhas dessincronizadas — um risco desnecessário.

Solução: codificar a posição (linha, coluna) como um único inteiro `linha * n_colunas + coluna` e usar `divmod(posicao, n_colunas)` para recuperar (linha, coluna) ao desempilhar. Uma única `Pilha(capacidade, int)` passa a bastar, já que a classe só aceita um tipo básico por instância.

2. Validar só ao desempilhar exige "adivinhar" a capacidade

A versão ingênua empilha os quatro vizinhos de qualquer célula processada, sem checar se ainda são alvo — a validação (limites e tipo de célula) só acontece quando a posição é desempilhada. Isso significa que uma mesma célula pode ser empilhada várias vezes antes de ser marcada como preenchida, e não há como calcular de antemão quantas operações de empilhar vão ocorrer. A versão ingênua contorna isso com uma capacidade

"estimada" ($n_linhas * n_colunas * 4$) — no teste de estresse com uma sala de 100×100, essa estimativa deixou apenas ~4% de folga em relação ao uso real (38.417 operações contra uma capacidade de 40.000), o que mostra que o fator $\times 4$ não é uma garantia, é um palpite.

Solução: validar o vizinho (`limites e matriz[l][c] == alvo`) *antes* de empilhar, e marcá-lo como preenchido no mesmo instante. Cada célula é então empilhada no máximo uma vez, o que permite dimensionar a Pilha com a capacidade exata $n_linhas * n_colunas$ — comprovadamente suficiente, sem estimativas.

3. Matriz como lista de strings força reconstrução a cada escrita

Strings são imutáveis em Python. A versão ingênua do algoritmo recursivo lê o arquivo como uma lista de strings e escreve uma célula com `matriz[linha] = matriz[linha][:coluna] + "0" + matriz[linha][coluna+1:]` — ou seja, reconstrói a linha inteira a cada célula preenchida, um custo proporcional ao número de colunas por escrita, em vez de custo constante.

Solução: representar cada linha como `list(str)` (lista de caracteres), permitindo `matriz[linha][coluna] = "0"` em tempo constante.

4. Recursão sem proteção não é uma opção viável para regiões grandes

O rascunho bruto do algoritmo recursivo não considera o limite padrão de recursão do Python (1000 chamadas). Isso não é um problema teórico: os testes de estresse mostram que uma simples sala vazia de 60×60 (3.600 células) já é suficiente para estourar `RecursionError` com esse rascunho, porque o preenchimento de uma área aberta tende a produzir uma cadeia de chamadas recursivas quase do tamanho da própria região (o algoritmo só "desempilha" — retorna — depois de esgotar os quatro vizinhos de cada célula visitada).

Solução: executar a busca recursiva em uma thread dedicada, com `threading.stack_size()` aumentado (64 MiB) e `sys.setrecursionlimit()` ajustado ao tamanho da matriz antes de iniciar a recursão, restaurando os dois valores ao final. Essa é uma mudança estrutural, não cosmética: sem ela, a rotina recursiva não atenderia ao enunciado para os exemplos maiores fornecidos (o labirinto de 30×70 já usa boa parte do limite padrão).

Ressalva sobre concorrência: `threading.stack_size()` e `sys.setrecursionlimit()` são configurações globais do processo, não por thread. A implementação restaura os dois valores em um bloco `finally` antes de a função devolver o controle, o que é seguro desde que

`preenchimento_recur_sivo.preencher` não seja chamada concorrentemente por múltiplas threads ao mesmo tempo — o que é verdade no uso previsto (CLI e interface gráfica de um único usuário, um preenchimento por vez). Fica registrado como uma limitação conhecida, não como um bug oculto.

5. Mecanismo de exibição acoplado ao algoritmo

O rascunho bruto não separava "calcular o preenchimento" de "mostrar o preenchimento". Isso dificultaria reaproveitar a mesma lógica de exibição por P passos entre os dois algoritmos e a interface gráfica, como pedido explicitamente pelo grupo.

Solução: os dois algoritmos finais têm a mesma assinatura e devolvem o mesmo contrato — a lista ordenada de posições preenchidas (`list[tuple[int, int]]`) — e apenas alteram a matriz recebida em memória. `reprodutor.py` (texto) e `interface_grafica.py` (janela) consomem esse contrato sem saber qual algoritmo o produziu.

Complexidade

Para as duas implementações finais, com n = número de células da matriz:

- `preencher` (recursivo e com pilha): $O(n)$ no tempo — cada célula é visitada e marcada no máximo uma vez.
- Espaço: $O(n)$ no pior caso — a pilha explícita ou a pilha de chamadas recursivas podem chegar a conter todas as células da região preenchida.
- A versão ingênua da pilha tem a mesma ordem de complexidade assintótica, mas com uma constante até 4× maior (medido nos testes de estresse) por reempilhar posições inválidas ou já preenchidas.

Resultado da revisão

A versão final é assintoticamente igual à ingênua ($O(n)$ em tempo e espaço), mas com uma constante menor, sem risco de estouro de recursão nas escalas testadas, sem depender de uma capacidade "estimada" para a pilha, e com o mecanismo de exibição desacoplado do algoritmo — o que era um requisito explícito do grupo, não apenas uma melhoria de estilo.